Step 2 — Saved State and a Human Pause Before Anything Goes Out — Hints

Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — What a pause really needs
- Hint 2 — The plan, and almost the whole thing

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — What a pause really needs

`interrupt(value)` inside a node **pauses** the graph (Doc09). The graph returns early, and the result contains `"__interrupt__"`; `result["__interrupt__"][0].value` is what you passed in.

To continue, call `graph.invoke(Command(resume=answer), config)`. Inside the node, `interrupt(...)` then **returns** `answer`.

Two things make this work:

- a **checkpointer** — where the paused state is saved; without one, there's nowhere to pause to;
- a **thread id** in `config = {"configurable": {"thread_id": "..."}}` — which saved run you mean.

Put the pause in the Approval agent's own file now — `agents/approval_agent.py` — because Step 5 will grow this same node:

```
approval_node(state):
    reasons = ["every answer is checked by a person for now"]
    decision = interrupt({"answer": state["answer"],
                          "failed_checks": reasons})
    "yes"         -> approved, path += ["human_approved"]
    anything else -> not approved,
                     answer = "Not approved — nothing was sent.",
                     path += ["human_rejected"]
```

`answer_node` does **not** change — the node reviews its `answer` field. Add `approved` to the state.

`build_graph(checkpointer=None)` takes the checkpointer as a parameter (Doc09's Build Task, Approach 2) and uses `MemorySaver()` only if none is given.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

```
main():
    with SqliteSaver.from_conn_string("checkpoints.sqlite") as cp:
        graph = build_graph(cp)
        loop:
            thread id = "run-" + current time + "-" + counter
            result = graph.invoke(start_state(task), config)
            if "__interrupt__" in result:
                ask_human(request)   # prints answer + reasons, asks
                result = graph.invoke(Command(resume=decision), config)
            print path and answer
```

Add `checkpoints.sqlite` to `.gitignore`.

Prove the state really comes from disk with **two scripts** (Doc09 showed `MemorySaver` fails this):

- `scripts/pause_demo.py` — fixed thread id `"demo-1"`, invoke, see the pause, **exit**.
- `scripts/resume_demo.py` — a new process: open the same SQLite file, build the same graph, `saved = graph.get_state(config)`. If `saved.next` is empty, nothing is paused. Otherwise ask yes/no and resume.

Run them one after the other. If the second one finishes the run, your state survived a restart.

Tests: give `run()` an optional `decision` (resume with it if the run paused; `None` = leave it paused), build the graph with `MemorySaver()`, and add cases 2-4 for an action task: it pauses; "yes" approves; "no" answers "Not approved".

Full solution: Show me the solution